PARTE 1:

SOCKETS

Ejercicios:

1. Código fuente del servidor (server.c). Identificar la secuencia de funciones de sockets utilizadas en la implementación del servidor. ¿Qué información debe proporcionarse en cada uno de los pasos?

La vida del socket sigue exactamente el estándar POSIX:

socket(): Crea el endpoint de comunicación.

- Información proporcionada: Dominio o familia (PF_INET para IPv4), tipo de socket (SOCK_STREAM para TCP) y protocolo (0).

bind(): Asigna una dirección y un puerto fijos al socket para que los clientes sepan dónde llamar.

- Información proporcionada: El descriptor del socket (welcomeSocket) y una estructura sockaddr_in configurada con la familia (AF_INET), el puerto (htons(7891)) y la IP local (inet_addr("127.0.0.1")).

listen(): Marca el socket como pasivo, poniéndolo a la escucha.

- Información proporcionada: El descriptor del socket y el tamaño máximo de la cola de conexiones pendientes (5).

accept(): Bloquea el servidor hasta que llega una petición, extrayéndola de la cola y creando un nuevo socket dedicado solo a esa comunicación.

- Información proporcionada: El socket de escucha (welcomeSocket), una estructura para guardar los datos del cliente (serverStorage) y su tamaño.

send() y recv(): Funciones para leer y escribir en la red.

- Información proporcionada: El nuevo descriptor de socket (newSocket), un buffer de memoria (buffer), la longitud a leer/escribir y flags (0).

close(): Se llama tanto para cerrar el socket dedicado al cliente tras servirle, como para cerrar el de escucha general al apagar el servidor.

2. Código cliente (client.c). ¿Cómo se conecta al servidor? ¿Qué información debe conocerse? Identificar la secuencia de funciones de sockets utilizadas en la implementación del cliente.

¿Cómo se conecta y qué debe conocer? El cliente se conecta de forma activa al servidor utilizando la función connect(). Para lograrlo, necesita conocer obligatoriamente la dirección IP del servidor (que en nuestro código se pasa como argumento al ejecutar: argv[1]) y el puerto en el que escucha el servidor (que en nuestro código está escrito a fuego en la línea 30: htons(7891)).

Secuencia de funciones:

1. socket(): (Igual que en el servidor).
2. connect(): Inicia la conexión (Three-way handshake TCP) pasándole la IP y el puerto configurados en una estructura sockaddr_in.
3. recv() y send(): Dentro de un bucle while para mandar las operaciones ingresadas por teclado y recibir los resultados.
4. close(): Para finalizar la conexión al terminar.

3. Opcional. Ejecutar el comando netstat -a antes y después de ejecutar el servidor. ¿Cómo se detecta que el servidor está escuchando? ¿Qué información especificada en el fichero fuente se observa?

-

4. Discutir características de sockets en términos de transparencia de ubicación, transparencia de acceso, facilidad de implementación para servidor y cliente y describir servicios en los que el empleo de sockets sería adecuado y servicios en los que no sería la opción más apropiada. Completar la tabla.

Transparencia de ubicación: Nula

- Explicación: El cliente tiene que saber de antemano la dirección IP y el puerto exactos del servidor y especificarlos en el código o en la llamada. Si el servidor cambia de máquina, el cliente deja de funcionar hasta que se actualice la IP.

Transparencia de acceso: Nula

- Explicación: Los datos se envían como secuencias de bytes planas. El desarrollador tiene que programar a mano el empaquetado (marshalling) y desempaquetado de los enteros (convirtiendo cadenas a enteros con atoi, gestionando buffers, etc.).

Facilidad de implementación servidor: Baja

- Explicación: Requiere mucho código repetitivo (boilerplate) para gestionar las estructuras de red, crear el socket, hacer el bind, el listen, y gestionar la cola con accept.

Facilidad de implementación cliente: Baja

- Explicación: Al igual que el servidor, hay que gestionar la conexión paso a paso y empaquetar/parsear la información enviada y recibida manualmente.

Interoperabilidad: Alta

- Explicación: Al ser el estándar base de comunicación en red (TCP/IP), puedes tener un cliente en C y un servidor en Python o Java y se entenderán perfectamente, siempre y cuando ambos respeten el formato de los bytes que se envían.

Servicios adecuados: Streaming y transferencia de ficheros

- Explicación: Es ideal para protocolos de bajo nivel, envío continuo de datos multimedia (vídeo/audio) o situaciones donde el rendimiento extremo justifique el esfuerzo de programar a tan bajo nivel.

Servicios menos apropiados: Lógica orientada a objetos / APIs complejas

- Explicación: Muy mala opción para servicios que requieran pasar estructuras de datos complejas, objetos o hacer llamadas a procedimientos (como esta misma práctica de la calculadora), debido al infierno que supone parsear todo a mano.

SUN RPCs

Ejercicios:

1. Observar los ficheros generados por rpcgen y comentar su funcionalidad en base a lo visto en las clases de teoría.

SUPUESTAMENTE HAY QUE EJECUTAR CÓDIGO

A partir de la interfaz .x, el comando rpcgen genera varios ficheros que automatizan la comunicación:

- int_library.h: El archivo de cabecera con la definición de las estructuras y los prototipos de las funciones para que cliente y servidor hablen el mismo idioma.
- int_library_clnt.c (Client Stub): Es el representante del servidor en la máquina cliente. El cliente lo llama como si fuera una función local, y este archivo se encarga de enviar los datos por la red.
- int_library_svc.c (Server Stub o Dispatcher): Es el representante del cliente en la máquina servidor. Se queda escuchando, recibe la petición de la red, la desempaqueta, llama a vuestra función de server.c y devuelve el resultado.
- int_library_xdr.c: Contiene las rutinas XDR (eXternal Data Representation) que transforman los datos locales al formato estándar de red (y viceversa) para que máquinas con distinta arquitectura se entiendan.

2. Definición de la interfaz (fichero int_library.x). ¿Por qué es necesario agrupar todos los parámetros de entrada en una estructura? Observar que es necesario indicar el número de programa, versión y procedimiento.

Por qué agrupar parámetros: Es necesario agrupar el tipo de operación y los operandos en la estructura argumento porque el estándar clásico de SUN RPC solo permite pasar un único parámetro a la función remota.
Identificadores: El servicio se identifica en la red gracias a tres valores exactos: el número de programa (0x00000007), la versión (1) y el número de procedimiento (1).

3. Ejecutar el comando rpcinfo -p antes y después de ejecutar el servidor. ¿Cómo se detecta que el servidor está escuchando? ¿Qué datos especificados en el fichero .x se reflejan?

SUPUESTAMENTE HAY QUE EJECUTAR CÓDIGO

Detección del servidor: Al hacer rpcinfo -p se le pregunta al portmapper de la máquina. Sabemos que el servidor está escuchando porque aparecerá registrado el programa con número 7 (o 0x00000007) asociado a los puertos TCP y UDP que el sistema operativo le ha asignado dinámicamente.

Datos reflejados: Se observa el número de programa (7), la versión (1) y el protocolo utilizado.

4. Código fuente del servidor (server.c). Observar que el registro del servicio en el portmapper es transparente para el programador, que solo debe preocuparse de implementar la funcionalidad del servicio. El código encargado de realizar el registro con el portmapper se encuentra en la función main del fichero int_library_svc.c. Identificar las partes de este código responsables de: - Crear sockets UDP y TCP. - Dar de alta el servicio en el portmapper. ¿Qué información debe proporcionarse? - Llamada a la función dispatcher encargada de procesar las llamadas del cliente. En la función dispatcher identificar para qué sirve el número de procedimiento declarado en el fichero .x. Localizar las partes de código encargadas del marshalling/unmarshalling de los argumentos y el envío de respuesta al cliente.

Aunque en server.c solo vemos la lógica matemática, la magia ocurre en el archivo autogenerado int_library_svc.c.

- Crear sockets: En el stub del servidor hay funciones como svcudp_create y svctcp_create que levantan los sockets automáticamente.
- Alta en el portmapper: Se realiza mediante llamadas a pmap_unset y svc_register, pasándole el número de programa, la versión y la función dispatcher.
- Dispatcher: Utiliza el número de procedimiento (en un bloque switch) para saber qué función ejecutar. Luego usa rutinas XDR para el unmarshalling de los argumentos, llama a procesapeticion_1_svc y hace el marshalling de la respuesta usando svc_sendreply().

5. Código cliente (client.c). ¿Cómo se conecta al servidor?¿Qué información debe conocer el cliente?

Conexión: Se conecta usando la función clnt_create().
Información necesaria: El cliente necesita conocer el hostname o IP del servidor, el número de programa (INT_LIBRARY_PROG), la versión (INT_LIBRARY_VERS) y el protocolo de transporte deseado ("tcp").

6. Discutir características de RPCs en términos de transparencia de ubicación, transparencia de acceso, facilidad de implementación para servidor y cliente y describir servicios en los que el empleo de RPCs sería adecuado y servicios en los que no sería la opción más apropiada. Completar la tabla.

Transparencia de ubicación: Media/Baja

- Explicación: Aunque no hace falta conocer el puerto exacto (porque de eso se encarga el portmapper), el cliente todavía necesita saber obligatoriamente la dirección IP o el nombre de host de la máquina donde reside el servidor para llamar a clnt_create.

Transparencia de acceso: Alta

- Explicación: El programador del cliente simplemente llama a la función procesapeticion_1() pasando sus parámetros, exactamente igual que si fuera una función local. El sistema oculta por completo la serialización (XDR) y el uso de los sockets de red.

Facilidad de implementación servidor: Alta

- Explicación: El programador se olvida de la red. Solo tiene que escribir la lógica de negocio en el fichero server.c (el switch con las sumas y restas) y devolver un puntero al resultado.

Facilidad de implementación cliente: Alta

- Explicación: Solo se requieren un par de líneas para crear el cliente (clnt_create) y otra para llamar a la función. No hay que gestionar colas ni buffers de memoria explícitos.

Interoperabilidad: Media

- Explicación: Soluciona el problema de diferentes representaciones de memoria (Big/Little Endian) gracias a XDR, pero está muy enfocado al ecosistema C/C++ y UNIX, siendo más difícil de integrar con lenguajes modernos sin librerías de terceros.

Servicios adecuados: Redes locales y entornos UNIX (ej. NFS)

- Explicación: Ideal para sistemas distribuidos en intranets donde se necesita rendimiento y se requiere invocar funciones remotas de forma nativa en C.

Servicios menos apropiados: Sistemas en Internet o multi-lenguaje

- Explicación: Muy mala opción para servicios expuestos a Internet, ya que la asignación dinámica de puertos del portmapper choca directamente con los cortafuegos (firewalls), y la interoperabilidad con lenguajes como Python o Java es farragosa.

7. Opcional. Modificar el prototipo de la función PROCESAPETICION para que en lugar de recibir una estructura reciba tres enteros indicando el tipo de operación y los dos operadores. Compilar el código. ¿Qué sucede? ¿Por qué?

-

CORBA

Ejercicios:

1. Abrir el fichero IntLibrary.idl que contiene la declaración de la interfaz y observar que se 8 declaran tanto el módulo como la interfaz. ¿Qué representaría el objeto remoto Corba y cuáles serían los métodos de este objeto?

Qué representaría el objeto remoto: El objeto remoto CORBA representa la instancia del servicio de operaciones matemáticas, es decir, nuestra "librería" de enteros alojada en el servidor.
Cuáles son sus métodos:

- procesarPeticion(in short type, in short op1, in short op2): Realiza el cálculo propiamente dicho.
- shutdown(): Un método etiquetado como oneway (asíncrono, no espera respuesta) para apagar el ORB remotamente.

2. Opcional. Observar los ficheros generados por los compiladores de interfaces idlj y omniidl. Comentar su funcionalidad en base a lo visto en las clases de teoría. Las siguientes referencias también pueden ser de utilidad: Implementación CORBA en Java: http://docs.oracle.com/javase/7/docs/technotes/guides/idl/jidlExample.html Implementación CORBA en Python: http://omniorb.sourceforge.net/omnipy3/omniORBpy/omniORBpy002.html

-

3. Fichero fuente servidor (server.java). Identifica y analiza los fragmentos de código responsables de: - Inicialización del ORB - Obtención del indentificador POA (Portable Object Adapter) raíz - Activación del POA manager - Creación de una implementación del servicio (servant) - Creación del objeto Corba y su asociación al servant - Localización del servicio de nombres - Registro de la librería de operaciones con números enteros en el servicio de nombres. - Espera de peticiones

Inicialización del ORB:
ORB orb = ORB.init(args, null);

Obtención del identificador POA raíz:
POA rootpoa = POAHelper.narrow(orb.resolve_initial_references("RootPOA"));

Activación del POA manager:
rootpoa.the_POAManager().activate();

Creación de una implementación del servicio (servant):
IntLibraryImpl iIImpl = new IntLibraryImpl();
Creación del objeto Corba y su asociación al servant:
org.omg.CORBA.Object ref = rootpoa.servant_to_reference(iIImpl);
junto con
IntLibrary href = IntLibraryHelper.narrow(ref);

Localización del servicio de nombres:
org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);

Registro de la librería en el servicio de nombres:
Se hace usando ncRef.rebind(path, href); tras definir el nombre con String name = "IntLibrary";

Espera de peticiones:
orb.run();

4. Fichero fuente cliente (client.py). Identifica y analiza los fragmentos de código responsables de: - Inicializar el ORB - Obtener una referencia al servidor de nombres. - Obtener una referencia al objeto Corba remoto a partir del nombre. - Llamada a la función principal de la librería.

Inicializar el ORB:
orb = CORBA.ORB_init(sys.argv, CORBA.ORB_ID)

Obtener referencia al servidor de nombres:
obj = orb.resolve_initial_references("NameService") y
rootContext = obj._narrow(CosNaming.NamingContext)

Obtener referencia al objeto Corba remoto a partir del nombre:

name = [CosNaming.NameComponent("IntLibrary","")],
luego se resuelve con
obj = rootContext.resolve(name)
y se adapta con
l = obj._narrow(IntLibraryApp.IntLibrary)

Llamada a la función principal:
result = l.procesarPeticion(int(sys.argv[1]), int(sys.argv[2]), int(sys.argv[3]))

5. Discutir características de Corba en términos de transparencia de ubicación, transparencia de acceso, facilidad de implementación para servidor y cliente y describir servicios en los que el empleo de Corba sería adecuado y servicios en los que no sería la opción más apropiada. Completar la tabla comparativa.

Transparencia de ubicación: Alta

- Explicación: El cliente no necesita saber ni la IP ni el puerto del servidor final donde reside el objeto matemático. Solo necesita saber el nombre del objeto ("IntLibrary") y conectarse al servicio de nombres (Naming Service). Si el servidor cambia de IP, simplemente se vuelve a registrar en el Naming Service y el cliente sigue funcionando igual.

Transparencia de acceso: Alta

- Explicación: El acceso es totalmente transparente. En Python, el cliente llama a l.procesarPeticion(...) exactamente igual que si estuviera llamando a un método de una clase local en Python. CORBA (a través del IIOP) se encarga de empaquetar y desempaquetar las variables por debajo.

Facilidad de implementación servidor: Media

- Explicación: Aunque automatiza la red, hay que escribir bastante código preparatorio (boilerplate) para arrancar el ORB, gestionar el POA (Portable Object Adapter), interactuar con el servicio de nombres, crear las referencias y generar el archivo IDL.

Facilidad de implementación cliente: Media/Alta

- Explicación: El código cliente (sobre todo en Python) es relativamente corto, pero sigue requiriendo localizar el ORB, resolver el servicio de nombres y hacer "narrowing" (adaptar la referencia genérica a la específica de vuestro objeto).

Interoperabilidad: Muy Alta

- Explicación: Es el rey histórico de la interoperabilidad en la época. Permite que un cliente escrito en un lenguaje interpretado como Python hable perfectamente con un servidor corriendo en una Máquina Virtual Java, gracias a que el contrato se define en el lenguaje neutral IDL.

Servicios adecuados: Sistemas empresariales heterogéneos y legados

- Explicación: Es útil en empresas muy grandes donde conviven sistemas bancarios antiguos en C++, middleware en Java y clientes en otros lenguajes, y todos necesitan compartir objetos remotos de forma estandarizada e independiente de la plataforma.

Servicios menos apropiados: Aplicaciones web y microservicios modernos

- Explicación: Es una arquitectura muy pesada y rígida comparada con soluciones modernas como las APIs REST con JSON o gRPC. Los cortafuegos de internet suelen bloquear su protocolo (IIOP), y la complejidad de configurar el ORB no compensa para servicios sencillos.

JAVA RMI

Ejercicios:

1. Abrir el fichero intLibrary.java que contiene la declaración de la interfaz y observar que la interfaz extiende la interfaz java.rmi.Remote y el método remoto lanza excepciones del tipo java.rmi.RemoteException.

¿Por qué extiende java.rmi.Remote?
Es una interfaz "marcadora". Sirve para indicarle a la Máquina Virtual de Java (JVM) que los métodos definidos ahí van a poder ser invocados desde otra máquina virtual diferente a través de la red.

¿Por qué lanza java.rmi.RemoteException? A diferencia de una llamada a un método local que casi nunca falla de forma inesperada, una llamada por red puede fallar por muchas razones externas (se corta el cable, el servidor se apaga, hay mucho lag). Java obliga a lanzar y capturar esta excepción para que el programador gestione obligatoriamente los posibles fallos de conexión.

2. Fichero fuente servidor (server.java). Observar que la clase remota se define como una implementación de la interfaz intLibrary definida en intLibrary.java. Comenta brevemente la funcionalidad de cada una de las instrucciones de la función main.

La clase se define como una implementación de intLibrary. Estas son las instrucciones clave de la función main:

- server obj = new server();: Crea la instancia local del objeto matemático (el servant que realmente hace las sumas y restas).
- intLibrary stub = (intLibrary) UnicastRemoteObject.exportObject(obj, 0);: Exporta el objeto al entorno RMI para que escuche peticiones en un puerto dinámico (el 0

indica que el sistema asigne uno al azar) y genera el stub (el "representante" que viajará por la red).
- Registry registry = LocateRegistry.getRegistry();: Obtiene la referencia al registro RMI local (el rmiregistry que arrancasteis en la consola).
- registry.bind("intLibrary", stub);: Publica o vincula el stub del servicio en el registro bajo el nombre o alias "intLibrary", para que los clientes puedan buscarlo.

3. Fichero fuente cliente (client.java). Comenta brevemente la funcionalidad de cada una de las instrucciones de la función main.

Estas son las instrucciones de la función main del cliente:

- Registry registry = LocateRegistry.getRegistry(host);: Se conecta al registro RMI ubicado en la dirección IP o nombre de host del servidor.
- intLibrary stub = (intLibrary) registry.lookup("intLibrary");: Le pregunta al registro remoto por el objeto llamado "intLibrary". El registro le devuelve el stub y hacemos un cast a nuestra interfaz intLibrary.
- short response = stub.procesarPeticion(...);: Invoca el método remoto. Fíjate que se llama exactamente igual que si el objeto estuviera en el propio ordenador del cliente; la red se oculta totalmente.

4. Discutir características de Java RMI en términos de transparencia de ubicación, transparencia de acceso, facilidad de implementación para servidor y cliente y describir servicios en los que el empleo de Java RMI sería adecuado y servicios en los que no sería la opción más apropiada. Completar la tabla.

Transparencia de ubicación: Media/Alta

- Explicación: El cliente necesita conocer la IP del rmiregistry (el servidor de nombres), pero no necesita saber en qué puerto exacto está escuchando el objeto matemático. El registro RMI se encarga de darle esa ubicación por debajo.

Transparencia de acceso: Alta

- Explicación: Inmejorable. El cliente invoca stub.procesarPeticion(op1, op2) como si fuera una clase local de Java estándar. El proceso de marshalling (serialización de los datos Java) lo hace la JVM automáticamente.

Facilidad de implementación servidor: Alta

- Explicación: Son apenas 4 líneas en el main para exportar el objeto y publicarlo en el registro, sin preocuparse por descriptores de sockets ni hilos de escucha.

Facilidad de implementación cliente: Alta

- Explicación: Basta con hacer un lookup al registro y llamar al método. Muy intuitivo y limpio.

Interoperabilidad: Baja

- Explicación: Es una tecnología diseñada en exclusiva para Java. Es obligatorio que tanto el cliente como el servidor se ejecuten en una Máquina Virtual de Java, por lo que no es apta para conectar, por ejemplo, un cliente en C con un servidor Java (ahí usaríamos CORBA o Sockets).

Servicios adecuados: Sistemas distribuidos 100% Java

- Explicación: Ideal para aplicaciones corporativas internas (intranets) donde toda la arquitectura tecnológica está construida utilizando el ecosistema Java (por ejemplo, arquitecturas J2EE antiguas) y se busca desarrollo rápido y orientado a objetos.

Servicios menos apropiados: Entornos heterogéneos o multi-lenguaje

- Explicación: Totalmente inapropiado si el sistema cliente y servidor están en lenguajes distintos, o si se necesita hacer una API pública para Internet (donde hoy en día se usa REST/JSON).

TABLA COMPARATIVA

	Sockets	**RPCs**	**CORBA**	**Java RMI**
Transparencia de ubicación	Baja / Nula	Media/Baja	Alta	Media / Alta
Transparencia de acceso	Baja / Nula	Alta	Alta	Alta
Facilidad implementar servidor	Baja	Alta	Media	Alta
Facilidad implementar cliente	Baja	Alta	Media / Alta	Alta
Interoperabilidad	Alta	Media	Muy Alta	Baja
Servicios adecuados	Streaming y transferencia de ficheros	Redes locales y entornos UNIX	Sistemas empresariales heterogéneos y legados	Sistemas distribuidos 100% Java
Servicios menos apropiados	Lógica orientada a objetos / APIs complejas	Sistemas en Internet o multi-lenguaje	Aplicaciones web y microservicios modernos	Entornos heterogéneos o multi-lenguaje

PARTE 2:

Como nuestra tecnología es Java RMI, la clave es proponer un servicio donde se necesite enviar objetos complejos de un lado a otro.

Si usamos Sockets o RPC para enviar un array dinámico o un objeto con muchos campos, es un dolor de cabeza. Pero en Java RMI es facilísimo gracias a la serialización nativa.

Propuesta completa de un **Sistema de Gestión de Préstamos de Biblioteca**, estructurada exactamente en las 3 diapositivas nos piden.

Diapositiva 1: Propuesta del Servicio

- Nombre del servicio: Sistema de Gestión de Préstamos de Biblioteca.
- Descripción: Un sistema donde un servidor centralizado mantiene el catálogo y el estado de los libros (disponible o prestado). Los clientes (que serían los ordenadores de los bibliotecarios o los terminales de consulta) se conectan para realizar dos operaciones principales:
    1. Buscar libros por autor o género.
    2. Registrar el préstamo de un libro mediante su código ISBN.
- Modelo de datos: Se creará una clase auxiliar llamada Libro (que implemente java.io.Serializable) con atributos como título, autor, ISBN y estado.

Diapositiva 2: Justificación del mecanismo de transporte

- ¿Por qué la arquitectura cliente-servidor con Java RMI es la idónea?
    - Paso de Objetos Complejos: Al buscar un autor, el servidor puede devolver directamente una lista de objetos (ArrayList<Libro>). Si usáramos Sockets, tendríamos que convertir toda esa lista a una cadena de texto enorme, enviarla y volver a separarla en el cliente. RMI lo hace transparente.
    - Excepciones Nativas: Si un usuario intenta sacar un libro que ya está prestado, el servidor RMI puede lanzar una excepción propia (ej. LibroNoDisponibleException). El cliente la captura con un bloque try-catch clásico de Java.
    - Ecosistema Homogéneo: Asumiendo que toda la red de la biblioteca utiliza equipos con la Máquina Virtual de Java instalada, no necesitamos la complejidad adicional ni la interoperabilidad de lenguajes que ofrece CORBA.

Diapositiva 3: Principales cambios en el código proporcionado

Siguiendo las instrucciones de indicar los cambios en líneas generales (sin programarlos), habría que adaptar los tres ficheros:

1. En la interfaz (intLibrary.java):
    - Cambiar el nombre de la interfaz a algo como GestorBiblioteca.
    - Eliminar el método matemático y declarar los nuevos métodos remotos, por ejemplo:
      ArrayList<Libro> buscarLibro(String autor) throws RemoteException;
      boolean prestarLibro(String isbn) throws RemoteException;
2. En el servidor (server.java):
    - Implementar la nueva interfaz GestorBiblioteca.
    - Cambiar el switch matemático por una estructura de datos en memoria (ej. un HashMap<String, Libro>) que actúe como base de datos del catálogo.
    - En la función main, cambiar el nombre de registro de "intLibrary" a "Biblioteca" al hacer el registry.bind().
3. En el cliente (client.java):
    - Actualizar la conexión para buscar el objeto remoto bajo el nuevo nombre: registry.lookup("Biblioteca").
    - Modificar la lectura de los argumentos (args) para que en lugar de recibir enteros, reciba comandos como "BUSCAR Tolkien" o "PRESTAR 12345".
    - Manejar la respuesta del servidor (ej. iterar sobre el ArrayList recibido para imprimir en pantalla los títulos de los libros).